# Die Palettenreihe im Lager

Selection Sort — das Kleinste auswählen und nach vorn holen

**AUSGANGSSITUATION**



Ahmed macht sein Praktikum im Getränkelager. Acht Paletten stehen in einer Reihe, jede trägt eine **Tournummer**. Für die Auslieferung müssen sie aufsteigend geordnet stehen. Mit dem Hubwagen kann Ahmed immer nur zwei Paletten die Plätze tauschen lassen – jeder Tausch dauert **zwei Minuten**. Die Lagerleiterin: „Schauen kostet nichts, fahren kostet Zeit. Fahr so wenig wie möglich." Mit dem Nachbarn-Verfahren aus der letzten Stunde müsste Ahmed diese Reihe 15-mal tauschen.

**Wie oft muss Ahmed höchstens tauschen, wenn er geschickt vorgeht – und braucht er dafür auch weniger Vergleiche als beim Bubble Sort?**

**Was tippst du?** Kreuze an, bevor du sortierst:

- ☐ weniger Tauschvorgänge und weniger Vergleiche
- ☐ weniger Tauschvorgänge, gleich viele Vergleiche
- ☐ gleich viele Tauschvorgänge

Die Palettenreihe vor dem Sortieren – Tournummern (gesetzte Annahme)

| Platz | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| Tournummer | 41 | 17 | 58 | 23 | 66 | 12 | 35 | 29 |

## 1 Aufgabe 1 — Aufwärmen

**Nenne** aus dem Gedächtnis die Anzahl der Vergleiche, die Bubble Sort aus der letzten Stunde bei acht Elementen macht.

**SO GEHT'S**

### So sortiert Ahmed mit wenig Fahrten: Selection Sort

1. Der **sortierte Teil** links ist am Anfang leer, der unsortierte Teil ist die ganze Reihe.
2. Suche im unsortierten Teil das **kleinste** Element – durch Anschauen, von links nach rechts.
3. Tausche es mit dem ersten Element des unsortierten Teils. Steht es schon dort, ist kein Tausch nötig.
4. Der sortierte Teil ist um eins gewachsen. Wiederhole ab Schritt 2, bis nur noch ein Element übrig ist.

BEISPIEL

### Beispiel mit vier Zahlen

7 3 9 1: Durchgang 1: kleinstes ist 1 (Platz 3), Tausch mit Platz 0 → **1** 3 9 7 (3 Vergleiche, 1 Tausch). Durchgang 2: kleinstes von 3 9 7 ist 3 – steht schon vorn, kein Tausch (2 Vergleiche). Durchgang 3: kleinstes von 9 7 ist 7, Tausch → **1 3 7** 9. Fertig: 6 Vergleiche, 2 Tauschvorgänge.

## 2 Aufgabe 2

**Vervollständige** die Tabelle: Trage je Durchgang die kleinste Nummer des unsortierten Teils, die Reihe danach und ein, ob getauscht wurde. Der erste Durchgang ist eingetragen.

| Durchgang | kleinste | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | Tausch? |
|---|---|---|---|---|---|---|---|---|---|---|
| Start | – | 41 | 17 | 58 | 23 | 66 | 12 | 35 | 29 | – |
| 1 | 12 | 12 | 17 | 58 | 23 | 66 | 41 | 35 | 29 | ja |
| 2 | | | | | | | | | | |
| 3 | | | | | | | | | | |
| 4 | | | | | | | | | | |
| 5 | | | | | | | | | | |
| 6 | | | | | | | | | | |
| 7 | | | | | | | | | | |

Tauschvorgänge insgesamt: _____ Vergleiche insgesamt (7 + 6 + … + 1): _____

## 3 Aufgabe 3

**Erkläre**, warum Selection Sort in jedem Durchgang höchstens einmal tauscht – obwohl es im Durchgang viele Vergleiche gibt.

**Satzanfänge:** „Beim Vergleichen merkt sich Ahmed nur …“ · „Erst wenn …, dann …“

MERKE

### Zwei Arten von Arbeit

Ein Sortierverfahren verursacht **Vergleiche** (schauen) und **Tauschvorgänge** (bewegen). Bubble Sort und Selection Sort vergleichen gleich oft: $n \cdot (n - 1) / 2$. Beim Tauschen unterscheiden sie sich: Selection Sort tauscht höchstens $n - 1$ mal, Bubble Sort bei jedem Vergleich, bei dem der linke Wert größer ist – im schlimmsten Fall ebenfalls $n \cdot (n - 1) / 2$ mal.

## 4 Aufgabe 4

**Bestimme** für Ahmeds Palettenreihe die Vergleiche, die Tauschvorgänge und die Fahrzeit (zwei Minuten je Tausch) – für Bubble Sort aus der letzten Stunde und für Selection Sort aus Aufgabe 2.

**Bubble Sort:** Vergleiche _____ Tauschvorgänge _____ Fahrzeit _____ min

**Selection Sort:** Vergleiche _____ Tauschvorgänge _____ Fahrzeit _____ min

## 5  Aufgabe 5 — Partnerarbeit

**Dokumentiere** einen eigenen Sortierlauf: Schreibe fünf verschiedene Zahlen in beliebiger Reihenfolge auf und sortiere sie mit Selection Sort, Durchgang für Durchgang. Dein Partner prüft, ob in jedem Durchgang wirklich das kleinste Element des Rests gewählt wurde.

| Durchgang | kleinste | 0 | 1 | 2 | 3 | 4 | Tausch? |
|---|---|---|---|---|---|---|---|
| Start | | | | | | | |
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |
| 4 | | | | | | | |

Tauschvorgänge: _____ Vergleiche insgesamt: _____ ☐ vom Partner geprüft

INFO

### Bausteine lesen und zusammensetzen

Der Code merkt sich beim Suchen nur den **Index** des bisher kleinsten Werts in `kleinster` – bewegt wird nichts. `range(start + 1, n)` liefert die Indizes hinter `start` bis zum Ende.

Erst nach der inneren Schleife wird einmal getauscht, mit der **Hilfsvariablen** `merker` wie in der letzten Stunde. Eine Zeile mit Doppelpunkt öffnet einen Block: was dazugehört, steht eine Stufe tiefer.

## 6 Aufgabe 6 — Code bauen

**Implementiere** die Funktion `selection_sort(liste)` aus den 8 Bausteinen: Bringe sie in die richtige Reihenfolge und trage im Bauplan je Zeile den Buchstaben ein – in der Spalte seiner Einrückungsstufe. Jeder Baustein wird genau einmal gebraucht.

**A**
```
kleinster = start
```
Index des bisher kleinsten Werts

**B**
```
return liste
```
fertig sortierte Liste zurückgeben

**C**
```
def selection_sort(liste):
```
Funktion: die Liste, die sortiert werden soll

**D**
```
if liste[i] < liste[kleinster]:
    kleinster = i
```
noch kleiner gefunden: Index merken

**E**
```
for start in range(n - 1):
```
start = erster Platz des unsortierten Teils

**F**
```
merker = liste[start]
liste[start] = liste[kleinster]
liste[kleinster] = merker
```
kleinsten Wert nach vorn tauschen

**G**
```
n = len(liste)
```
Anzahl der Elemente

**H**
```
for i in range(start + 1, n):
```
den Rest des unsortierten Teils durchsuchen

Bauplan – Einrückungsstufe (eine Stufe = 4 Leerzeichen)

| Zeile | Stufe 0 | Stufe 1 | Stufe 2 | Stufe 3 | Stufe 4 |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | | | | |

**Wenn du nicht weiterkommst:** Womit beginnt jede Funktion? Welche Zeilen enden mit einem Doppelpunkt – und was muss darunter eine Stufe tiefer stehen?

## 7 Aufgabe 7 — Schreibtischtest

**Teste** dein Programm aus Aufgabe 6 auf dem Papier mit `liste = [3, 1, 2]`: Trage für jeden Durchgang der äußeren Schleife ein, welcher Index am Ende in `kleinster` steht, ob getauscht wird und wie die Liste danach aussieht.

| start | kleinster am Ende | getauscht? | Liste danach |
|---|---|---|---|
| 0 | | | |
| 1 | | | |

Rückgabewert: _____ Vergleiche insgesamt: _____



Diese Aufgabe hat mehr als eine richtige Antwort – es kommt darauf an, **was im Lager teuer ist**. Begründe deine Wahl mit den Zahlen aus Aufgabe 4.

## 8 Aufgabe 8 — Ahmeds Entscheidung

**Beurteile** mit deinen Zahlen aus Aufgabe 2 und 4, welches Verfahren Ahmed nehmen soll – und ob er damit auch beim Schauen Zeit spart. Vergleiche am Ende mit deinem Tipp vom Anfang.

**Satzanfänge:** „Ahmed sollte …, weil er dann nur … statt … tauscht." · „Beim Vergleichen spart er …, denn …"

## 9 Aufgabe 9 — Vertiefung

**Übertrage** das Verfahren auf die Bestenliste des Schulturniers: Die höchste Punktzahl soll auf Platz 0 stehen. Schreibe die Zeilen der Funktion auf, die sich ändern, und gib der Variablen einen passenden neuen Namen.

## 10 Aufgabe 10 — Zusatz

**Entwickle** einen Plan für das Lagerprogramm: Es verwaltet 3000 Artikelnummern, und jeden Tag wird etwa 500-mal geprüft, ob eine Nummer existiert. Halte fest, welche Verfahren aus den letzten vier Stunden das Programm in welcher Reihenfolge nutzen soll, und schätze die Vergleiche pro Tag mit und ohne deinen Plan.

### PRÜFE DICH SELBST

- [ ] Wächst in deiner Tabelle der sortierte Teil links in jeder Zeile um genau eine Nummer?
- [ ] Ist deine Zahl der Vergleiche für Selection Sort dieselbe wie für Bubble Sort?
- [ ] Gibt deine Funktion 12 als erste und 66 als letzte Nummer aus?